

Frontend Routes

- **/:** Landing route for signup page. The user enters credentials, which they will use in the login page to login.
- **/login:**
 - Purpose: User authentication UI.
 - Actions:
 - Login: POST to backend with username, password.
 - Error handling: Shows "Invalid username or password" on 401; other errors show generic login failure.
- **/home:**
 - Purpose: Display global stats and highlights.
 - API calls:
 - GET most watched animes
 - GET top average rated animes
 - GET least watched animes
 - GET least average rated animes
- **/dashboard:**
 - Purpose: User-specific data: watchlist, watched, ratings.
 - API calls:
 - GET user watchlist
 - GET user watched anime
 - GET per-anime rating for current user (batched per anime)
- **/animelist:**
 - Purpose: Browse all anime; card back shows average rating only if 2+ users rated.
 - API calls:
 - GET all anime
 - GET anime-with-multiple-ratings (IDs + avg rating where userCount ≥ 2)
 - The search bar is always there. We check for filtering only on /animelist. Navigations between pages clear search and are disregarded.

Backend API — User

Base: /user

- **POST /user/register**
 - Purpose: Create account
 - Request: JSON { "username": string, "password": string } (password ≥ 8 chars)
 - Responses:
 - 200: JSON User (created)
 - 409: Username exists; body mirrors request or { message: "You already have an account" }
 - 400: { message: "Incorrect username or password." } or validation errors
- **POST /user/login**
 - Purpose: Authenticate user
 - Request: JSON { "username": string, "password": string }
 - Responses:
 - 200: JSON User (authenticated) and client typically stores session info locally
 - 401: { message: "Invalid username or password" }
 - 400: { message: "<error details>" } on service exceptions

- **POST /user/watchlist/add**

- Purpose: Add anime to watchlist
- Request: JSON { "userId": number, "animeId": number }
- Responses:
 - 200: No body required; client updates local state
 - 4xx: Validation or not found

- **POST /user/watchlist/remove**

- Purpose: Remove anime from watchlist
- Request: JSON { "userId": number, "animeId": number }
- Responses:
 - 200: No body
 - 4xx: Validation or not found

- **GET /user/watchlist/{userId}**

- Purpose: Get user's watchlist
- Response: JSON UserAnimeWatchlist[] (objects include userId, animeId, addedDate)

- **POST /user/rating/set**

- Purpose: Set rating
- Request: JSON { "userId": number, "animeId": number, "rating": number } (0–10 range; float)
- Responses:
 - 200: No body
 - 4xx: Validation errors

- **GET /user/rating/{userId}/{animeId}**

- Purpose: Get a user's rating for a specific anime
- Response: JSON UserRating or empty/404 depending on implementation
- Status:
 - 200: Found (object or null payload)
 - 404: Optional if not found

- **POST /user/watched/set**

- Purpose: Mark anime watched/unwatched
- Request: JSON { "userId": number, "animeId": number, "watched": boolean }
- Responses:
 - 200: No body
 - 4xx: Validation errors

- **GET /user/watched/{userId}**

- Purpose: List watched anime for a user
- Response: JSON UserAnimeWatched[] (objects: userId, animeId, watched)

- **GET /user/stats/most-watched**

- Purpose: Aggregated counts across users
- Response: Array of maps { [animeId]: count }
- Note: Only includes animes that are watched by 2+ users, shown on frontend "Most Watched"; no overlap with least watched

- **GET /user/stats/top-rated**

- Purpose: Top average rated animes (avg $\geq$ 6)
- Response: Array of maps { [animeId]: avgRating }
- Notes: Sorted by rating descending; ties handled by ID as secondary

- **GET /user/stats/least-watched**

- Purpose: Bottom of watch counts excluding most watched anime
- Response: Array of maps { [animeId]: count }
- Note: Includes animes only watched by 1 user watched, shown on frontend "Least Watched"; no overlap with most watched

- **GET /user/stats/least-rated**

- Purpose: Worst average rated animes (avg < 6), excludes top-rated
- Response: Array of maps { [animeId]: avgRating }
- Notes: Sorted by rating ascending; no overlap with top-rated

- **GET /user/stats/anime-with-multiple-ratings**

- Purpose: Animes with ratings from 2+ distinct users
- Response: Array of objects { animeId: number, avgRating: number, userCount: number }
- Notes: Used by /animelist to show card back average if userCount >= 2

Backend API — Anime

Base: /anime

- **GET /anime**

- Purpose: Fetch all anime
- Response: JSON Anime[] (includes id, title, status, totalEpisodes)

- **GET /anime/title/{title}**

- Purpose: Fetch single anime by title
- Response:
 - 200: JSON Anime
 - 404: Not found

- **POST /anime/watched/update**

- Purpose: Update watched status
- Request: JSON { "userId": number, "animeId": number, "watched": boolean }
- Responses:
 - 200: JSON UserAnimeWatched or null if no change
 - 4xx: Validation errors

Auth & Interceptors

- Frontend uses auth guard and interceptor to protect routes and attach session info if applicable. Routes are protected when the user is not logged in.
- Login flow stores userId and username in session storage; guard checks these for access into other pages.

Error Handling

- 409 Conflict on duplicate username during registration
- 401 Unauthorized on invalid login (username or password)
- 400 Bad Request for validation errors (ex: password is less than 8 characters, username or password is blank)